# 计算机组成原理 · Cache 与快表 · 考点精讲与大题整合

408 各科大题知识点强化课 | 课程第 6 讲（分 P8） | 时长 3 小时 25 分 11 秒

**本讲范围：**两条线 —— Cache（存储层次 → 块与行 → 标记的本质 → 有效位 → 三种映射方式与地址结构 → 替换算法与替换算法位 → 写策略与脏位 → 总容量 → 命中率 / 缺失率 → 访问流程）与 **快表 TLB**（页表 / 二级页表 / 快表的查询流程 → 快表的三种映射方式 → 快表与 Cache 的完全平行性 → 两道大题）。老师说「关于 Cache 这块一定要总结一下，就是要从这五个角度去学习」，本资料就是按这五个角度重排的。

**读法建议：§3 方法论**（拆地址两步、三种映射对照、总容量五件套、命中率两法、答题模板）→ **§4**（他全程手推的直接映射例题 + 例 3.4 + 30 题）→ **§5**（两道大题：Cache 与快表）→ **§7 速查卡**。§2 是把「为什么」讲透的一段（标记为什么存商不存块号、为什么块内地址人人都有），§6 是易错点。

**一句话：**「Cache 这边其实你只要把知识点学会，题目其实并不是很灵活」「就是知识点可能比较琐碎，要把它学透」——本讲值钱的是**地址结构为什么这么切**，不是背公式。

## 目录


1 考情分析：Cache 的分量与五个考察角度

2 核心概念（把「为什么」讲透）

3 解题方法论：拆地址、总容量、命中率

4 典型题型与例题（含老师全程手推的例题）

5 本讲两道大题精解（Cache 与快表）

6 易错点清单

7 考场速查卡

附 本资料的来源与边界


ASR 校正说明（读引用前先看这一段）

转写稿由本地 Whisper 生成，同音错字密度极高且几乎全砸在术语上（**Cache 一个字被转成十几种写法**）。引用处已统一校正，主要替换：**开始 / 开水 / 快标 / Kass / Cast / CASH / 車輪 / 超市 / 快表（该是 Cache 时）→ Cache**；**组存 / 主神 → 主存**；**跨 / 框 / 炮 / 泡 / 胖 / 矿 / 垮 / 胯 / 库 / 匡 → 块**；**框类 / 矿内 / 矿泪 / 跨类 / 胖类 / 液内 / 叶内地址 → 块内地址**；**硬射 / 应设 / 用手 / 订设 / 应社 → 映射**；**主相连 / 组项链 / 祖相连 / 组监 / 主乡连 / 组象连 / 组下零 → 组相联（奥陆 / 澳陆 / 奥路 = 二路）**；**全箱连 / 全项链 / 全香联 / 全江连 / 权相联 → 全相联**；**取于 / 取域 / 取语 / 取渔 → 取余**；**双 / 伤 / 熵 → 商**；**与数 / 余素 / 云朔 / 余朔 → 余数**；**杭号 / 航号 / 杭数 / 航数 → 行号 / 行数**；**标击 / 标几 / 表击 → 标记**；**有效慰 / 优异 / 有消费 / 有效为 → 有效位**；**双位 / 占位 / 张位 → 脏位**；**权写法 → 全写法、威胁法 → 回写法、止血 → 直写**；**替换散瓦尾 / 替换三法 / 听话设法位 → 替换算法（位）**；**糯葛二 / 糯糕2 / 怒缩 / LOG2入属 → $\log_2$**；**夹载 → 加载、掉进来 / 吊进来 → 调进来、确实 / 却失 / 缺业 → 缺失、片一辆 / 偏一辆 → 偏移量**；**叶号 / 夜号 / 业号 → 页号、叶表 / 夜表 / 液表 → 页表（项）、叶框号 / 夜框号 / 叶匡浩 → 页框号**；**出移 / 出一 → 除以、结取 / 解取 / 撷取 → 截取**；**大体 / 真体 / 整体 → 大题 / 真题、技术 → 计组、考评 → 考频、内水 → 内存、主间 → 组间、仿存 → 访存、客家 → 课下**。

**数字类误转不在上表展开**（「624 字节」= 64 字节、「32 分之 1」= 1/32、「十二的六十八次」= 2 的 6 次方 = 64 次），逐条列在 §附的「转写含糊处」清单里。

## 1 考情分析：Cache 的分量与五个考察角度

「首先他的考试频率比较高。」「就是（Cache）的话，这边**总共是 10 道大题**。」「书上的话是放了这个 8 道，**就 3.5 和 3.6 放了 8 道**，然后还有两道是放在了其他位置，放到了 **4.3 和 5.6 的位置**。」「408 他命题的话，有时候会把两道题目的题干是相关的……为了方便做题，他是把两道（Cache 的）放到其他位置，**总共 Cache 加起来的十道**。」「所以是（计组）里面**最重要的一个知识点**」「就是**大题考频最高的**。」
「但是虽然他的考频最高，**但是他的考法也是比较最固定的**。」「一般来说经过一定的练习之后**分数还是比较好拿的**。」「但是呢这个 Cache 很多人可能会做题，题目应该也都能做出来，**但是很多地方的理解还是可能是不太够**」「就说你还是要专门刻意地针对（Cache）再好好听听一下。」

**他给的复习提纲：Cache「一定要总结一下」（下面五个角度是他点的，本资料按此重排）**

「关于 Cache 这块一定要总结一下，就是要从这几个角度去学习：**第一个是映射方式**，无论是直接映射，还是全相联还是组相联，原理都要清楚；**第二种要学习的就是总容量**，Cache 要算总容量；**第三个的话就是命中率**，命中率问题；**然后第四个的话就是访问过程**，Cache 的访问流程，或者问你这个是否命中、如何判断是否命中；**第五个是考比较器的个数和位数**。一般历年的（真题）主要是从这几个角度去考察的。」

```
① 映射方式   → 判断是直接 / 全相联 / 组相联；写地址结构；求标记位数          §2.6 §3.1–3.2 §5.1(1)
② 总容量     → 每行 = 数据 + 标记 + 有效位 + 脏位 + 替换算法位，再乘行数      §2.9 §4.2(1)
③ 命中率     → 给一段代码，问命中率 / 缺失率 / 缺失次数                      §2.10 §3.3 §4.3 §5.1(2)
④ 访问过程   → 给一个地址，说明是否命中、命中后怎么取数、缺失怎么办          §2.5 §3.4 §4.1 §4.2(3) §5.1(3)
⑤ 比较器     → 直接 1 个；全相联等于行数；组相联等于路数；位数 = 标记位数     §2.11
```

**课尾原话：这一讲怎么用**

「相对来说，**Cache 这边其实你只要把知识点学会，题目其实并不是很灵活**」「就是**知识点可能比较琐碎，要把它学透**，稍微多总结一下。」「这一块它的考法其实很固定，大题的考法很固定，但是说做题目的时候可以稍微多想一点。」「**千万不要因为这边大题考法特别固定就过于轻视，它的频率很高**，所以一定要把考过的那些大题完全把它做透。」
下一讲预告（原文）：「等到下次上课的话，就讲那个第四章……寻数据的指令、指令的寻址和一些（变址）那些大题。」

**本讲实际覆盖了两块，不止 Cache：**① Cache（§1–§4，占约四分之三）；② **快表 TLB**（§2.12、§3.5、§5.2）。他明确说快表这块是**必须补**的：「快表的组相联……其实书上没有专门讲，但是出现在真题里面出现挺多的」「这一块王道书上解释的不多，但是（在）真题里面还是挺高频的，所以一定要注意一下」。

## 2 核心概念（把「为什么」讲透）

### 2.1 存储层次：书上 82 页图 3.2

> 「（从这个）光盘到磁盘，磁盘就是外存，然后到主存，到 Cache，再到寄存器，再到 CPU，**速度是越来越快的**」「然后它的这个成本……往上这个价格也是越来越高，**这成本也是越贵的。**」「**Cache 和主存是能和 CPU 交换信息的**」「CPU 是能直接访问主存，**CPU 也能直接访问 Cache**」「（主存和辅存也能交换信息）」「**辅存必须通过主存**」「就外存的东西必须先放到主存，然后之后才能被 CPU 去（访问）。」

### 2.2 Cache 的依据：局部性原理

> 「其实 Cache 的话，**就是把内存的一部分数据放进去**」「就把内存里面比较常用的一部分数据放到 Cache」「**Cache 里面只有内存的一部分数据。**」「然后 Cache 的依据是因为**局部性原理**」「因为局部性原理才能用这个 Cache。」
> 「比方说这个局部性原理就是**最近你用到的一些数据，未来可能再次被用**」「所以你某一次用到的一些数据，你把它加载到 Cache，那下次再用的话 Cache 就能命中了。」

> ⚠ **他只说了「局部性原理」这四个字**
>
> 全讲**没有出现「时间局部性」「空间局部性」这两个名称**（已逐字检索，0 次），也没有讲 Cache 的容量、块长、映射方式如何利用局部性。本资料不替他补这一课。

### 2.3 块、行、块长；以及「以字为单位」和「以块为单位」

> 「（书上 117 页）**Cache 和主存会被划分为（大）小相同的块**」「（Cache）的块也叫做 **Cache 行**」「**每块由若干字节构成，这个长度叫块长或者行长**」「Cache 是**仅保存主存中最活跃的一些副本。**」

```
CPU 和 Cache 之间交换是以【字】为单位
Cache 和内存之间交换是以【块】为单位
```

> 「**他们真正去交换数据……不是以这个小的单位，而是以这个大的单位，直接把整个这一大部分放到 Cache 里面**」「他们交换是以块为单位。」「CPU 和 Cache 是以字，它的意思就是：**CPU 并不是说这些数据都要用到，它只会用到这一个**」「CPU 用到这一小个数据，**它会把整个块都调进来了**，调进来之后 CPU 依然是只用这一小个。」

**对应关系（他反复画的那张图）：**CPU 给出一个物理地址 → 先按块大小切开 → 高位是块号、低位是块内地址 → 块号决定「放到 Cache 的哪一行 / 哪一组」（直接映射要再除以行数，组相联要再除以组数）→ 块内地址决定「最后取块里的哪一个字」。

### 2.4 标记的本质：为什么存「商」而不存「块号」

> 「这个 Cache 的块数是比主存要少」「然后所以只需要把一部分信息放到 Cache —— Cache 的**每一块增加一个标记位**」「**标记位注意它的作用是说明它是内存中哪一块的副本。**」
> 「然后标记的内容，其实他说相当于主存中的块的编号，**但是这里其实说的不太准**，待会我们再（解释）为什么不太准。」

```
场景：Cache 有 4 行，块大小 4 字节
   内存 0 号块 ÷ 4 = 商 0 余 0  ┐
   内存 4 号块 ÷ 4 = 商 1 余 0  ┘ 余数相同 → 都只能放到 Cache 的第 0 行
   ⇒ 光看余数（行号）分不清放的是谁，必须再记一个东西

最直观：直接存块号 0 / 4 → 但「这个 4 起码要占 3 位」，浪费空间
计算机的做法：只存【商】 —— 存 0 就是 0 号块，存 1 就是 4 号块（能反推回块号）
```

「零号块可以放到 Cache 的零行，你 4 号块也能放到零行」「这个计算机肯定得知道……**他到底 Cache 的零行放的是内存的零还是内存的 4。**」「你最直观的方法，直接把 0 和 4 这个数字存进去……但这样会浪费空间。」「**因为它们两个余数相同，所以都会放到 Cache 的零行，但是它们两个的商不一样**」「所以这里只需要把这个商存进去：**存 0 代表是内存的 0 号块，存 1 就代表内存放的是 4 号块。**」「因为这个 4 起码要占 3 位……现在只把它们的商放进去，明显要更省空间一点。」

⚠ **全讲最容易混的一点：有【两个】标记**

「注意标记是**两个标记**：你**地址结构里面这个标记**，和 **Cache 里面存的那个标记**，它是两个东西」「虽然他们的位数什么都一样，**但是含义是不一样的**，要把它们区分开。」

· **地址结构的标记**：从 CPU 给出的那个物理地址里切出来的一段，是「我要找谁」。

· **Cache 里的标记**：Cache 每一行自己存的一段，是「我这里现在放着谁」。

· 命中 ⟺ 两者**相等**（并且有效位为 1）。算总容量时加的那个标记，是**Cache 里存的那个**；地址结构里那个只决定它的**位数**。

## 2.5 有效位：为什么光标记相等还不够

「这个有效位它就是**字面意思，用来说明这个 Cache 里面的数据是否是生效的。**」「**Cache 命中的前提有两个：一个是（地址结构的）标记相等，然后还得有效位为一。**」

### 他给的场景（写操作只写了主存、没改 Cache）：

「假设现在 CPU 要对内存进行修改……把内存的 3 号（块）UK 改成了一个 P」「**CPU 进行写操作只把它写到内存了，Cache 没改**，那这时候你会发现内存的块和 Cache 的话就不一样了」「所以这时候需要把 Cache 的这个有效位给它**置为零。**」

「那如果现在 CPU 又想去访问 3 了……通过行号定位到零这一行，会发现**标记也相等**，现在的标记是 0，Cache 里面的标记也是 0，他就会认为现在 Cache 里面放的就是他想要的」「**但是现在有一个有效位也为 0**」「里面虽然是你想要的那个块，**但是这个数据可能是比较旧的**，你没有（对）Cache 更改，这有效位为零的话**也不能算命中。**」

```
命中条件（两把锁，缺一不可）
   ① CPU 地址切出来的【标记】 == Cache 该行存的【标记】
   ② 该行的【有效位】 == 1
两个都满足 → 命中；否则 → 未命中（缺失），要走调入 / 替换流程
```

## 2.6 三种映射方式：定义、地址结构、公式

| 方式 | 老师原话 | 地址结构（高位 → 低位） | 公式 |
| --- | --- | --- | --- |
| **直接映射** | 「直接映射的特点就是**每一块装入到 Cache 的唯一位置**」「他的特点就**无需使用替换算法**。」 | `标记 \| 行号 \| 块内地址`<br>「直接映射的地址结构也是要记得」 | `行号 = 主存块号 mod 行数`<br>`标记 = 主存块号 ÷ 行数（商）` |
| **全相联** | 「全相联的话是**每一块可以放到任何位置**」「这些就没有什么要注意的。」 | `标记 \| 块内地址`<br>「只有两部分：块内地址和标记」 | `标记 = 主存块号（整个）` |
| **组相联** | 「组相联的话注意这边有这样一句话：**组间采用的是直接映射，组内采用的是全相联**，然后**每组的块数叫多少路组相联**。」 | `标记 \| 组号 \| 块内地址`<br>「这个地址结构也要记」 | `组号 = 主存块号 mod 组数`<br>`标记 = 主存块号 ÷ 组数（商）` |

> 「所以这个公式要记，因为有的题目会用到这些公式，这些公式都是（理解性的）。」「Cache 块号（Cache 行号，**行和块是一个意思**）等于主存块号对行数取余」「**直接映射是对行数取余、组相联是对组数取余。**」

**为什么全相联的标记要「整个块号」**：「你怎么区分这个 Cache 的零行到底是放的 0 还是 1 还是 2 还是 3 还是 4？」「**那就只能把整个内存块号放进来。**」「全相联的话就相当于它**直接拿内存块号当标记了。**」

**反过来看直接映射**：「**直接映射只是把这个商放进来当标记**」「它是能通过这个标记反推这个块号的」「所以直接映射是只把一部分放进来了，**只把这个块号的一部分作为标记。**」

**一句话总结（本资料据他原话概括）**：映射方式的本质，就是**把内存块号拆开**——直接映射拆成「标记 + 行号」，全相联整个块号就是标记，组相联拆成「标记 + 组号」。这一句在快表那儿会原封不动地再用一次。

## 2.7 替换算法与替换算法位

> 「**如果是全相联的话就要有替换算法**」「比方说现在这个 Cache 的 4 个块都用（满）了……那现在如果你想往 Cache 里面放这个第 5 个块……**你只能从这四个里面挑一个置换出去**」「那到底挑哪一个……这时候就（涉及）到替换算法。」
> 「**如果直接映射就不需要替换算法**，因为直接映射任何一个内存块最终放到 Cache 的哪一行它是直接能算出来的，**就不存在多个被选项。**」「正是因为它有多个选择，才要（用）替换算法。」

> 「这个（真）题上用的最多的往往是 LRU」「代表这个**最久未使用**」「如果需要发生置换，他把这个最久没用的替换掉。」「这个替换算法位……**看映射方式**」「**如果是直接映射是不需要替换算法位；如果全相联的话应该是 $\log_2$ 行数；如果组相联的话是 $\log_2$ 路数。**」
> 「如果是全相联的话……任何一个 Cache 行都能被放进去，**现在一共有四个选择可以选，那你需要两位。**」「如果是组相联，你通过计算能得出它的组号……**它放到哪一组是（确定的）**……但是放到某一组到底哪个位置不确定，**所以应该是 $\log_2$ 路数。**一组有两个，那备选项只有两种情况，通过一个二进制位就够了。」「8 路组相联，一组八个……那就应该 $\log_2 8$，**$\log_2 8$ 就是三位。**」

> ⚠ **替换算法他只讲了 LRU 一种**
>
> 全讲**没有出现 FIFO / 先进先出 / 随机（RAND）**中的任何一个（已逐字检索，0 次）。本资料不替他补教材上的其余替换算法。

## 2.8 写策略与脏位

> 「这里还有一个概念叫**全写法**和**回写法**。」「**全写法就是说，如果你要对内存的数据进行更改，你把 Cache（和）内存都进行修改**」「这个叫全写法。」「还有一个叫回写法，**回写法就是你只对 Cache 面去更改**」「比方说你要更改，只把 Cache 面进行修改，把 Cache 面改，内存先不改。」
> 「什么时候（写回）？Cache 的数据被一个新的数据……**这个置换的时候**」「你在覆盖之前要把（这个块）直接更改、同步到这个内存」「**先只在 Cache 里面改，什么时候 Cache 元素就要被替换的时候，然后再立马把它写回到内存**。」「（有的地方）会叫**直写**，有的会叫**全写**，就是同时写回内存和 Cache」「**直写意思就是说没有脏位**。」

> **脏位（= 修改位 = 一致性维护位）**：「（如果是回写）你通过 Cache 里面去记录一下……**如果记录一个 1，说明这里面发生修改了**，那之后被置换的时候，他应该把这个修改同步到内存」「**如果这里的这个脏位是零，那说明这里没有发生修改**，他和内存里面本来就一样了……那所以之后要置换的时候，内存就不用管。」「（脏位）有时候也叫做**修改位**，然后有时候也会叫做**一致性维护位**。」

| 写策略 | 行为 | 脏位 |
| --- | --- | --- |
| **全写法**（又叫直写） | 改数据时 Cache 和主存**同时改** | **没有**脏位（「直写意思就是说没有脏位」） |
| **回写法** | 只改 Cache；**被替换出去时**才写回主存 | **有 1 位**：1 = 改过、置换时要写回；0 = 没改、不用管 |

> ⚠ **他只讲了写策略，没有讲写分配 / 非写分配**
>
> 全讲**没有出现「写分配」「非写分配」**（已逐字检索，0 次）。他也没有讲写缓冲、写直达与写回对总线流量的影响。本资料不补。

## 2.9 Cache 总容量：五个部分，考试不给

> 「顺着刚才的把这个 Cache 的总量给写完」「**这个 Cache 的总量除了数据部分，然后还有标记**（有 4 个）」「**这些都是要记得，考试是不会给有效位、脏位、替换算法位**。」「数据的话往往就是这个块的大小」「但是要注意单位，**往往题目上给的单位都是字节，然后这个标记位给的单位是比特**」「**这个标记要根据地址结构去算**」「有效位是一定有的，占一位」「脏位，回写的话是有一位，全写的话是没有」。

```
Cache 总容量 = 行数 × 每行容量
每行容量 = 数据块      （= 块大小，题目给的是【字节】，要 ×8 换成比特）
         + 标记        （位数由【地址结构】决定：直接/组相联取商那段，全相联取整个块号）
         + 有效位      （1 位，必有）
         + 脏位        （回写 1 位；全写 0 位）
         + 替换算法位（直接映射 0 位；全相联 log₂行数；组相联 log₂路数）
```

⚠ 全部统一成【比特】再相加，最后答案的单位是比特

## 2.10 命中率与缺失率：最容易埋坑的一段

> 「命中率……往往这种题目也是比较好做的」「但是这个点其实也有些东西需要注意的。」「Cache 和内存是以块为单位，你访问第一个块的第一个数据会缺失，缺失之后**它会把整个块调进来，那后面三个就都命中了**」「那它的**缺失率就是 1/4，命中率就是 3/4**。」

> **坑在这里：**「因为有的题目，每一个元素可能除了读还会写」「你如果要执行这样的一句代码，说明首先你要先访问这个内存把 A[i] 这个元素读取出来，读取完之后你还要（再写）一个，**还要再写回到这个内存**」「这样的话就相当于**一个元素是访问两次的，就读一次写一次**」「要去分开看一下。」

**他给的三种改法（同一个数组，只改代码，缺失率就变；算式见 §4.3）：**`A[i] = A[i] + 32`（读一次 + 写一次）→ **1/8**（他念「八分之一」）；`A[i] = 32`（他说的「复制语句」，只写）→ **1/4**；`B[i] = A[i] + 32`（他说的「放到另一个变量」，只读）→ **1/4**。

> **用「局部代表整体」的前提（他专门强调）：**「但是这样做有一个前提要去验证」「你会发现这个题目一千个元素，总共是一千个数组元素，然后每一个 Cache 放四个，**它是刚好放了整数个 Cache 块——250 个块**。」
> **不占整数个块就只能硬算：**「像现在这个题目它没有占整数个，前面这个占了一半，后面这个也是占了一半多一点……**你就不能按照之前的方法去做**」「之前那种方法你用局部代表整体，你只能是每个块情况都一样……才能去做」「**现在的话……你只能用总的缺失次数除以总的访问次数，就是缺失率；总的命中次数除以总的访问次数，就是命中率**。」

## 2.11 比较器的个数与位数（第五个考察角度）

> 「所以这个比较器的个数……**如果是直接映射，一个就够了；全相联它应该等于行数；如果是组相联的话等于路数**。」
> 「因为直接映射……是行号的话，**它只会找到 Cache 的某一行了，找到某一行之后只和这一行去比就行了**。」「全相联……**每一个行都有可能是放着你想要的那个块**」「**会把每一行都去比一下**，就等于总共的行数。」「组相联，你通过组号提前就定位到某一组了，**但这一组具体是哪一个会命中不清楚的，得都比完才知道**」「现在每组两个需要两个比较器，每组有四个就是四个比较器。」
> 「**但这个位数的话始终就是等于标记的位数**」「这个比较器是干嘛用的？**就是把 Cache 的标记（和）我当前地址的标记进行比较的**，所以就等于标记的位数。」

> 比较器个数：直接映射 = 1 ；全相联 = 行数 ；组相联 = 路数（一组几块就几个）
> 比较器位数：恒等于【标记位数】（因为它比的就是标记）

## 2.12 快表 TLB：和 Cache 完全平行的一套

> 「像这个是操作系统里面的一个经典的过程：就给你一个页号，然后再给一个页内地址」「这个操作系统经常会让你把逻辑地址求成物理地址，转化到物理地址、地址变换」「或者有时候也叫虚拟地址」「**这个虚拟地址和逻辑地址一般不做区分**。」
> 「（有的地方）会把它叫做**页号和偏移量**」「有的地方会叫做**页号和页内偏移量**，然后有的地方把偏移量叫做**页内地址**。」
> 「然后有了页号之后**根据页号去查页表**」「（页表项）有的地方会叫做**物理页号**，有的地方会叫做**页框号**」「甚至有的地方会叫做这个**框号**，**这个框号就是指的页框号，不要和 Cache 那个块搞混了**。」

「然后只要得到一个页框号，**你拿页框号和逻辑地址的偏移量拼接，拼接就是最后的物理地址**。」「至于为什么拼接后就是物理地址的这个……这个等到操作系统到时候再展开来讲，现在的话就会用就行了。」

他说快表与 Cache「都是相通、完全通用」的（本资料概括为「平行」；他未用过这个词）

「这个一定要注意一下，就是**王道书上他没有专门去讲快表的这个映射**」「王道书上这个快表的这个映射方式他讲的不多。」

「首先注意 **Cache 针对的是物理地址，然后快表针对的是逻辑地址**」「然后他们都是（有）这个映射方式的：**直接映射、全相联、组相联**。」

「所以映射方式的本质，就是说**把内存块号去进行拆分**：直接映射拆成标记和行号，全相联整个内存块号就是标记，组相联拆成组号和标记。」「如果是快表的话，它后面就不叫块内地址了，**叫页内地址**……**无论是快表用的什么映射方式，前面通通都可以当成页号**。」

「**在快表和页表这个映射方式一定要多对比……放到快表里面都完全通用的，包括他们的地址结构都是通用的**。」「如果是 Cache 的话，左边都叫内存块号；如果是快表的话，左边都叫页号。」「**快表注意，它不属于内存**，因为页表都是在内存放的，**快表属于缓存这一（类）**。」

# 3 解题方法论：拆地址、总容量、命中率

## 3.1 十进制拆地址：两步除法（他花了一小时讲的就是这一步）

```
第①步   把【物理地址（Cache）/ 虚拟地址（快表）】除以【块大小 / 页大小】
          商    = 主存块号（快表：虚页号）          ← 低位是余数，高位是商
          余数 = 块内地址（快表：页内地址）

第②步   按映射方式继续除：
          直接映射：块号 ÷ 行数    → 商 = 标记 ，余数 = 行号
          组相联   ：块号 ÷ 组数    → 商 = 标记 ，余数 = 组号
          全相联   ：不用除，整个块号就是标记

第③步   拿行号 / 组号去 Cache 定位 → 比标记 → 命中则按【块内地址】取数
          （快表同构：把「块号」换成「虚页号」，「行数 / 组数」换成快表的行数 / 组数）
```

「你有了物理地址，**首先第一步你是要先除以块的大小**」「为什么要除以块的大小？**因为内存和 Cache 是以块为单位去放的**」「你不是说去算一下时（它）应该放在哪，**而是先求出它的块号 2，之后拿 2 去放**。」

「用这个物理地址先除以块的大小，得出一个商，**这个商的话就是块号；余数的话叫块内地址**」「**余数的意思就是：当前的 10 号在整个块里面它的相对位置是多少**。」

他专门停下来讲的「为什么要懂十进制」（很多人的盲区）

「为什么二进制这样拆分就能求出这些东西？**背后就是十进制这些运算**。」「（书）上学习 Cache，就把二进制的拆分……**把这些十进制这些计算（原理）屏蔽掉了**」「**所以很多人可能只会拆分，但是这个十进制的一些计算原理就不太清楚，或者不太熟练**」「（要）能把它们连贯起来。」

## 3.2 二进制拆地址：从低位往高位切

物理地址（或虚拟地址）的二进制，【从低位往高位】切三段：
低 $\log_2$(块大小) 位 → 块内地址（本次三个例子都是 6 位 / 12 位）
再高 $\log_2$(行数) 位 → 直接映射的行号 ┐
再高 $\log_2$(组数) 位 → 组相联的组号 ┘ 二者按映射方式二选一
剩下的全部高位 → 标记

本讲出现过的三组地址结构（已按他给的数逐条验算）：
例 3.4 ：256MB = $2^{28}$ → 28 位 ；块 64B → 6 位 ；8 行 → 3 位 ；标记 28−9 = 19 位 ✓
Cache 大题：主存 32 位 ；块 64B → 6 位 ；32KB÷64B = 512 行，8 路 → 64 组 → 6 位 ；
标记 32−12 = 20 位 ✓
快表大题 ：虚拟 30 位 ；页 4KB → 12 位 ；8 组 → 3 位 ；标记 30−15 = 15 位 ✓
虚页号 = 30−12 = 18 位 ✓

⚠ **唯一会翻车的地方：按字编址（他专门举了反例）**

「有可能出现这种题目的，**按字编址，假如说一个字是 32 位，32 位的意思就是四个字节**」「那这时候一个块是 64 个字节，**它其实只会占用 16 个地址**」「那相当于总共只有 16 个地址，这时候块内地址**就应该占 4 位了。**」
「所以这个块内地址一定是：**你先求出来它占多少个地址再取 $\log_2$。**」「在做 408 的题目凡是碰到**按字节编址**一般没什么（问题），你就正常去做就行；凡是碰到**按字编址一定要稍微细心一点，它能够出现错误的。**」

## 3.3 命中率 / 缺失率的两种算法（先判断用哪一种）

【方法一｜局部代表整体】适用：数据占满整数个块（从块边界开始、长度是块容量的整数倍）
缺失率 = 每块的缺失次数 ÷ 每块的访问次数
他讲的两个标准例子：
块 16B、数据 4B → 每块 4 个数据、只读一次 → 首访缺 1 次 / 4 次访问 → 缺失率 1/4
块 16B、数据 4B → 每块 4 个数据、读写各一次 → 首访缺 1 次 / 8 次访问 → 缺失率 1/8

【方法二｜硬算】适用：不占整数个块（前面占一半、后面占一半）
缺失率 = 总缺失次数 ÷ 总访问次数
总访问次数 = 元素个数 × 每个元素的访问次数（⚠「读一次 + 写一次」= 2 次）

【互推】缺失次数 = 总访问次数 × 缺失率 ；命中率 = 1 − 缺失率

## 3.4 访问流程的答题模板（第三问照这个写）

【命中】
① 按【行号】（直接映射）或【组号】（组相联）定位到 Cache 的某一行 / 某一组
② 把【当前地址切出来的标记】与【Cache 该行存的标记】比较
③ 标记相等 且 有效位为 1 → 命中
④ 再按【块内地址】从该行取出那一个具体的数据
⚠ 第④步不能省：「CPU 的目的只是为了取出其中一个块的某一个数据，不是为了整个块」

【缺失】—— 第 5 题 / 第 7 题这类「说明过程」的题照这个写
① 同上定位并比较：标记不等 或 有效位为 0 → 未命中
② 把当前地址所在的【整个块】从主存调入 Cache 的对应行（Cache 与主存以块为单位）

③ 把当前地址的【标记】写入该行的标记字段，并把【有效位】置 1
④ 再按块内地址取出数据（「刚放进来还要把有效位置为 1」「放进来还没有停」）

【特例】题干若说「初始时 Cache 为空」（原话：「题干上说了这个初始时 Cache 是空」）
→ 一定不会命中，直接写 ②③④ 即可。他原话：「没必要再这么回答……其实刚才那些答上去也不会错，只不过就相当多答了一些步骤。」

### 3.5 快表与 Cache 的完全对照（他说「多对比」）

| 对比项 | Cache | 快表 TLB |
|---|---|---|
| 针对的地址 | **物理地址** | **逻辑（虚拟）地址** |
| 低位字段 | 块内地址 | 页内地址（= 页内偏移量，两种叫法同义） |
| 高位字段 | 内存块号 | 页号（虚页号） |
| 里面存的是 | **主存的数据**（以块为单位整个搬进来） | **页表项**（一个页框号；页表项很小，常 4 字节） |
| 映射方式 | 直接 / 全相联 / 组相联 | **同样三种**（王道书没专门讲，真题里高频） |
| 定位公式 | 行号 = 块号 mod 行数；组号 = 块号 mod 组数 | 行号 = 页号 mod 行数；组号 = 页号 mod 组数 |
| 命中条件 | 标记相等 且 有效位为 1 | 标记相等 且 有效位为 1 |
| 拿到之后做什么 | 按块内地址取出那一个数据 | 拿页框号与页内地址**拼接**成物理地址 |
| 在哪 | 缓存，不属于内存 | 缓存，不属于内存（页表才在内存） |

## 4 典型题型与例题

下表题号是**老师手上讲义 / 课后题的编号**（例 3.4、30 题、第五题、第七题、15 题、20 题），**不是统考年份**。他本讲**亲口说出的年份是 0 个**，详见 §5.3。

### 4.1 直接映射手推例题（他逐笔算了 5 个地址，本资料按原话复现）

```
设定（他现场画的图）：块大小 = 4 字节（每块放 4 个数据）；Cache 共 4 行
      内存 4 号块 = A B C D    内存 0 号块 = H I G K    2 号块块内地址 2 处 = M

地址 ÷4商=块号 ÷4余=块内地址 块号mod4=行号 块号÷4商=标记 行内标记 结果
 10     2          2            2           0        空       缺失→调块2
 18     4          2            0           1       空(0行)    缺失→调块4
 17     4          1            0           1        1        命中→取B
  3     0          3            0           0        1        未中0≠1→换块0
  1     0          1            0           0        0        命中→取I

验算：10 = 4×2+2 ✓  18 = 4×4+2 ✓  17 = 4×4+1 ✓  3 = 4×0+3 ✓  1 = 4×0+1 ✓
      取数位置全部对得上：ABCD[2]=C ✓  ABCD[1]=B ✓  HIGK[3]=K ✓  HIGK[1]=I ✓
```

「这个 XYMN 是块里面的数据，**很多人刚学的时候会把那个地址和数据搞混，这个一定要避免**——比如这个 10 号是地址，10 号里面的数据是 M」「你 CPU 的目的肯定是把 10 号地址里面的数据取出来。」
「（访问 18 时）**你能算出 18、它的标记是 1**，所以它会把内存的 4 号块调到 Cache，并且会把标记 1 存进来」
「CPU 接下来还会根据块内地址，**块内地址是 2**，就会找到块内地址 2 的相对位置，把里面的 C 拿出来。」
「（访问 3 时）CPU 会发现 Cache 里面放的标记是 1，但是你要访问的物理地址的标记是零，**标记就不相等了**」
「**当前我要访问的物理地址所在的块，并不是 Cache 里面放的那个块**，虽然行号一样。」「**那就要发生置换了**……Cache 里面的标记就要改成 0 号块的标记，0 号块的标记是一个 0。」
「（访问 1 时）**当前这个物理地址的标记是零，Cache 里面放的也是零**，那说明 Cache 里放的就是你想要的那个块」「最后会根据块内地址 1 找到想要的数据，找到 l。」

## 4.2 例 3.4（书上例题，他要求「课下再做一遍」）

「这个例题还出的比较好的」「**这个三问刚好是不同的考察角度：第一问考察总容量，第二问考察你这个映射方式，第三问考察这个访问的流程。**」

已知：直接映射；Cache 8 行；块大小 64B；主存地址空间 256MB；按字节编址

(1) 总容量
主存地址位数：256MB = $2^{28}$ 字节，按字节编址 → $2^{28}$ 个地址 → 28 位
块内地址 = $\log_2 64$ = 6 位 ；行号 = $\log_2 8$ = 3 位 ；标记 = 28 − 6 − 3 = 19 位
每行 = 数据 64×8 = 512 位 + 标记 19 位 + 有效位 1 位 = 532 位
总容量 = 532 × 8 行 = 4256 位
⚠ 「4256」是本资料按他的口径算出的（他原话只说到「最后算出来的单位是比特」「用总行数乘以每行的总容量就行了」，没有念出结果）

(2) 把 3200 号主存地址拆开
① 十进制：3200 ÷ 64 = 商 50 余 0 → 块号 50、块内地址 0
直接映射：50 mod 8 = 2 → 行号 2 ；50 ÷ 8 = 商 6 → 标记 6
二路组相联（8 行 → 4 组）：50 mod 4 = 2 → 组号 2 → 落在第 4、5 行
② 二进制：把 3200 转成二进制，直接找中间那 3 位（行号）/ 2 位（组号）
⚠ 老师给了方法并说了二路组相联「余数是 2 → 行号对应 4 和 5」；行号 2、标记 6 是本资料按他给的算式补算的（他没念出这两个数）

(3) 给定主存地址，说明 Cache 访问过程
他念的地址是「0123456」，并明确说「先找到 001，先根据行号找到这一行」→ 行号 = 001
然后照 §3.4 的模板写：定位行 → 比标记 + 看有效位 → 命中则按块内地址取数；
不命中则把所在的【块】整个调入该行、写入标记、有效位置 1，再按块内地址取数
⚠ 这个地址的进制转写不清（十进制 123456 还是十六进制 0x123456）。
两种读法都验算过：123456 ÷ 64 = 1929 余 0，1929 mod 8 = 1 ✓；
0x123456 = 1193046，1193046 ÷ 64 = 18641 余 22，18641 mod 8 = 1 ✓
—— 两种读法的行号都是 001，与老师说的完全一致。本资料只用「行号 001」，不替他补出块内地址与标记。

「（第二问）比较直接的方法你是把 3200 转成二进制，转成二进制之后如果问你这个行号，你**找到中间的三位就行了。**」「如果是二路组相联，一共八个，每两个一组，**总共是四组，然后组号占两位**」「（余数是 2 的话）**它的行号应该对应的是这个 4 和 5。**」
「另外一种就通过十进制去做」「这个 Cache 和内存是以块为单位，**所以你第一步一定得先把它除以 64，先求出它**

> **的块号**」「先求出块号，那余数是一个块内地址」「直接映射，那你用块号直接除以这个行数，**这个商其实就是标记，余数就是行号**」「如果是组相联，那你应该是拿块号除以组数，**商是标记，余数是组号**。」

### 4.3 30 题（选择题）：命中率里的「读写各一次」陷阱

```
题：一个 Cache 块 16 字节，每个数据占 4 字节 → 一块放 4 个数据；
    数组共 1000 个元素；代码是把 A[i] 读出来、加 32、再写回原数组。

解：每个元素被访问 2 次（读 1 次 + 写 1 次）→ 1000 × 2 = 2000 次访问
    1000 ÷ 4 = 250 个块（刚好整数个块，前提成立 ✓）
    每块只有第 1 次访问缺失，其后 7 次全命中 → 250 次缺失
    缺失率 = 250 ÷ 2000 = 1/8          ← 他念的就是「八分之一」
    命中率 = 1 − 1/8 = 7/8

改题一：A[i] = 32（只有写）          → 1000 次访问、250 次缺失 → 缺失率 1/4
改题二：B[i] = A[i] + 32（只有读）  → 1000 次访问、250 次缺失 → 缺失率 1/4
```

> 「就一个块的话是 16 个字节，每个数据只占四个字节，**说明一个块能放四个数据**。」「先将 A[i] 进行一次读操作，从内存里面读完之后才能把它进行相加，加完之后还要再把它写回到这个数组」「**所以每一个数组元素相当进行了一次读操作和一次写操作**。」
> 「第一个元素在读的时候会缺失，缺失会把整个块调进来，**那接下来第一个元素的写和后面每一个元素的读写都会命中**」「它的缺失率应该是这个八分之一。」「**它是刚好放了整数个 Cache 块——250 个块**。」
> 「如果改成复制语句，把 32 复给 A[i]，相当只进行一次写操作，这时候它的缺失率就应该是四分之一。」「如果把 A[i] 加完 32 放到另一个变量里面，这时候相当只进行一次读操作。」

### 4.4 他给的「不占整数个块」的反例（方法二的由来）

```
他现场的残例：一共 3 个块、9 次访问，每个块都是第一次访问缺失
    → 缺失 3 次（「占了几个块就缺几次」）、命中 6 次
    → 缺失率 = 3 ÷ 9 = 1/3
⚠ 这段是边画边讲的，每块放几个数据他念得很快；能确认的只有「3 个块」「9 次」
  「缺失三次」这三个数，本资料不补图形细节
```

> 「第一个元素缺失的时候，其实 Cache 每占用几个块就会缺失几次」「**所以有几个块它就缺失几次**」「（总访问）总共是 1 2 3 4 5 6 7 8 9，总共是 9 次」「你用 9 减去缺失的就（是）命中的（次数）」「如果这个题目让你算缺失率，就应该是**缺失的次数除以总次数**。」

### 4.5 第五题（他只点了思路：「第一问让你算总容量、第二问让你算这个行号」）

```
题：已知数组首地址、按行优先存放、每个元素占若干个地址，求 A[0][31] 与 A[1][1] 的地址，
    再拿这个地址去求它落在 Cache 的哪一行。

行优先（先放第 1 行、放完再放第 2 行）：
    addr(A[i][j]) = 首地址 + ( i × N + j ) × 每个元素的大小
列优先（先放第 1 列、放完再放第 2 列）：
    addr(A[i][j]) = 首地址 + ( j × M + i ) × 每个元素的大小
      （M、N 为行数、列数）
```

求出物理地址后求行号，两条路都行：
① 转成二进制按地址结构切中间那几位　② 拿十进制直接去除
⚠ 他没念出 M、N 与元素大小，也没念出结果，本资料不补数值

「行优先就是先把第一行放完，按照行去放，放完第一行再放第二行」「任何一个下标应该等于首地址加上前面 i 行，每一行是 N 个，最后一行是这个（j 个）」「**一定要还要乘以每一个元素的大小**」「因为这边每一个元素是可能占这个多个地址的。」「如果按照列去放的话，先存第一列再存第二列，那前面应该有这（j）列……每一列是 M 个，最后一列是 i 个。」
「**这两个公式是数据结构里面的公式，但是说这个放到这里也是适用的。**」「通过这个公式你能求出 A[0][31] 和 A[1][1] 的这个地址」「只要求出这个物理地址，求出物理地址你去求行号，**一方面你可以转到二进制去拆分，另一方面拿十进制直接去除也都可以。**」

### 4.6 选择题 15 与 20：页表 vs 快表，就考「怎么找」

```
【15 题 | 用普通页表】页面大小 4KB → 低 12 位是页内偏移量，高位是页号
   坑：题里给了一个【十六进制】的 82，很容易当十进制 82 用
       0x82 = 8×16 + 2 = 130  ← 他念的就是这个换算
   流程：按页号查页表 → 得页框号（存在位为 1）→ 与页内偏移量拼接 → 物理地址

【20 题 | 用快表】快表是全相联 → 扣掉页内地址，前面【整段都是标记】
   流程：不查页号，直接把【当前虚拟地址的标记】与【快表里存的标记】逐项比
         → 标记相等 且 有效位为 1 → 命中 → 取出快表里的页框号 → 与偏移量拼接
   ⚠ 他读出的那个标记值转写为「03FFF」，数值不确定，本资料不写具体值
```

「首先一个页面大小是 4KB，那你把第 12 位就是偏移量，高位就是页号」「所以这里是给了 16 进制，这里给的是 10 进制，**这里是很容易踩坑，它是有干扰选项**」「可能有的人不小心就选了 82，误以为页号是 82」「**因为它是个 16 进制的 82，它就是 8×16 加 2，就是一个 130。**」
「（快表）是直接整个前面拿页号当标记的，**就扣掉页内地址剩下的全是标记**」「快表的话是直接去里面看，看一下有没有这个标记」「**标记相等，并且（当前虚拟地址有一个标记、快表里面本身也会存一个标记），如果他们两个相等并且有效位为一就命中了——其实和 Cache 的流程很像。**」「你无论（用）快表还是页表，**你的目的都是为了得到一个页框号。**」

## 5 本讲两道大题精解（Cache 与快表）

### 5.1 Cache 大题（他称「第七题」）：32 位地址 / 8 路组相联 / 32KB

```
题干要素（全是他现场念的）：主存地址 32 位、按字节编址；指令 Cache 与数据 Cache 分离；
     8 路组相联；采用直写（= 全写）；LRU 替换算法；Cache 块 64B；数据区容量 32KB；
     Cache 初始为空。

(1) 标记位数 / 替换算法位 / 修改位
    块内地址：按字节编址 → 64B 占 64 个地址 → log₂64 = 6 位
    行数 = 数据区容量 ÷ 块大小 = 32KB ÷ 64B = 2¹⁵ ÷ 2⁶ = 2⁹ = 512 行
    组数 = 512 ÷ 8（8 路）= 64 组 = 2⁶ → 组号 6 位
    标记 = 32 − 6 − 6 = 20 位
```

```
    LRU 位 = log₂8 = 3 位          （「这八个里面选一个」）
    修改位：直写 → 不需要（0 位）
```

```
(2) 缺失次数
    一个块放 64B ÷ 4B = 16 个数据；每个元素读写各一次 → 每 32 次访问缺 1 次 → 缺失率 1/32
    方法一：总访问次数 = 1024 × 2 = 2048 → 缺失次数 = 2048 × 1/32 = 64
    方法二：1024 ÷ 16 = 64 个块，每块缺 1 次 → 64 次
    ★ 两种方法都得到 64，互相印证 ✓
    前提验证：数组首地址（他念的末字节 C0 = 1100 0000）低 6 位全为 0
              → 块内地址 = 0 → 从块边界开始放、刚好占整数个块 ✓

(3) 说明取指令的访问过程 → 照 §3.4 模板；因「初始时 Cache 为空」，直接写缺失处理三步
```

**「这个 408 的大题绝大多数情况下，每一句话、题干的每一句话都是有用的」**「所以你在再去学 408 的大题，就是一定要注意：**每题干上的每一个信息是怎么用上去的**」「选择题有时候会给一些干扰选项，**大题的话绝大多数题目其实每个条件都是有用的。**」

**「凡是让你求标记的位数，你只能去求地址结构」**「组相联分成三个部分：块内地址、组号和标记。」「（数据区容量）他告诉你数据区的总容量，又告诉你一个块的大小，**用这个总容量除以一个块的大小，这样能得出一个 Cache 的行数**」「现在是 8 路组相联，那用 $2^9$ 再除以每组是 8 个，得出 $2^6$ 组，**所以组号占 6 位**」「**总共 32 位扣掉（这）12（位），（剩下）是标记，所以标记这 20 位。**」

「LRU 位一般指的是替换算法位……因为组相联你放到哪一组其实固定的……但**是每一组里面有 8 个，这 8 个里面放哪一个，不确定了**」「那这八个里面选一个呢，替换算法位就应该是 $\log_2 8$」「这样的话就是 LRU 位三位。」

「是否有修改位？**因为直写是不需要有修改位，只有回写才有修改位。**」

**（第三问的答法，他专门纠了一次）**「有人去回答第三问的时候他可能会这样去回答：就仿照刚才那个例题，他先说先通过组号……『（低）6 位是块内地址，中间六位是组号』……先通过组号找到零这一组，再看这一组里面的八个有没有标记是这个（值）并且有效位为一的，如果有就命中，命中之后根据块内地址去访问。」

**「没必要再这么回答，因为题干上说了这个初始时 Cache 是空」「初始时 Cache 空就意味着一定不会命中」**「其实刚才那些答上去也不会错，只不过就相当多答了一些步骤。」

「所以这个题目要重点去回答（缺失）处理过程：缺失之后应该是**把当前这个地址所在的块放到 Cache 的零这一组**，并且把**当前地址的标记写回到相应的 Cache 行**」「因为一组的话其实有八个 Cache 行，他放到哪一个了，你就把哪一个的标记写成这个标记」「写完之后，刚放进来还要把**有效位置为 1**」「放进来还没有停，因为 **CPU 和 Cache 是以字为单位进行交换**，放完之后还要根据这个块内地址把这个数据去访问出来。」

## 5.2 快表大题（他也称「第七题」）：30 位虚拟地址 / 二路组相联 / 8 组

```
题干要素：主存（物理）地址 24 位、按字节编址、采用分页；页面大小 4KB；
      快表二路组相联 + LRU；快表共 8 组；虚拟（逻辑）地址 30 位。

(1) 地址划分（照 Cache 组相联的地址结构搬）
    页内地址 = log₂(4KB) = 12 位（低 12 位）
    虚页号   = 30 − 12 = 18 位（高 18 位）

(2) 标记 / 组号（他说的「把虚页号的高位又拿着当作标记，低位拿着组号」）
    组号 = log₂8 = 3 位
    标记 = 30 − 12 − 3 = 15 位
```

```
(3) 给一串虚页号，问放到哪些组、最后一次怎么替换
    虚页号序列：10 → 12 → 16 → 7 → 26 → 4 → 12 → 20
    组号 = 虚页号 mod 8：
       10 mod 8 = 2   12 mod 8 = 4   16 mod 8 = 0   7 mod 8 = 7
       26 mod 8 = 2    4 mod 8 = 4   12 mod 8 = 4   20 mod 8 = 4
    过程：
       10  → 第 2 组（该组第一个位置）
       12  → 第 4 组
       16  → 第 0 组
        7  → 第 7 组
       26  → 第 2 组（该组已有 10，放第二个位置，二路正好放满）
        4  → 第 4 组（该组已有 12，放第二个位置）
       12  → 第 4 组，组内已有 12 → 【命中】
       20  → 第 4 组，两个位置已被 12 和 4 占满 → 淘汰一个
             12 刚被访问过 → 按 LRU 淘汰【4】，20 顶替进去
    ★ 结论：最后一次（20）替换掉的是 4

(4) 虚拟地址 30 位 → 32 位，快表表项增加几位
    页内地址 12 位不变（页面仍 4KB）；组号 3 位不变（仍是 8 组）
    → 只有标记变：15 位 → 17 位，增加 2 位
    快表表项 = 有效位(1) + 标记(15) + 页框号 → 有效位与页框号位数都没变
    → 【每个表项增加 2 位】
```

「第一问，你可以仿造 Cache 的组相联的地址结构」「这个页的大小是 4KB，所以页内地址占 12 位」「总共是 8 组，所以组号占 3 位」「题干上专门说了总共是 8 组，8 组（的话）这除数就是 8」「所以这里应该是 15 位：页内地址 12、（组号）3 位、然后标记是 15 位，总共是 30 位。」

「（第二问）其实他不说你也应该知道……**其实这句话你放到 Cache 也没问题：把内存块号的高位拿着当标记，低位拿着当组号。**」「因为无论这个 Cache 还是快表……**Cache 是把内存块号进行划分（各种映射），快表是把页号（按）各种映射方式去进行划分。**」

「如果组相联有这样一个公式：你 Cache 里面是满足**内存块号对组数取余等于组号**」「那同理你放到快表里面，它就是……**不是内存块号，应该是虚页号；虚页号对组数取余等于组号。**」

「因为这里题干上说了采用的是 LRU 替换算法，LRU 就是最久未使用」「**这个 12 刚用过，那最久未使用应该是 4**」「最后这个 20 会把 4 置换掉。」

「快表里面有什么东西看一下，**无非就是一个有效位、标记和页框号**」「唯一受到影响的就标记刚才变了」「**像页框号是不会变的，它里面放的页框号的位数并没有变**」「所以标记变了多少位，每一个表项就变了多少位。」

⚠ **这道大题里他念错过的两处（与算式对不上）**

① 他在推第一问时先念成「**总共虚拟地址 30 位，扣掉了 15 还剩 17 位是标记**」——30 − 15 = 15，不可能是 17；他紧接着自己改口为「**标记是 15 位，总共是 30 位**」，本资料按 **15 位**取。

② 他在讲第一问时说「如果单纯问虚页号，那就**高 20 位**」——30 − 12 = 18；他随后自己纠正为「**虚页号应该是 18 位，高 18 位的虚页号、低 12 位的页内地址**」，本资料按 **18 位**取。

③ 他念余数那串时最后读成「余 0」，但后面逐项分析明确说 12「是余四，还是放到四这一组」——12 mod 8 = 4，本资料按 **4** 取。

## 5.3 年份认定说明：本讲老师一个年份都没说

**结论：本资料一律不给这些题标年份**

> 本讲 3 小时 25 分的讲稿里，**「年」字只出现 5 次**，且没有一次是在说某道题的年份：
> 「但是呢**近几年**他那个 Cache（有的题目会出的比较难）」「一般**历年**的（真题）主要是从这几个角度去考察的」「因为很多大题的话，它都是参考**往年**的大题改了改重新去出的」「（想一下）其他**年份**有没有类似的考法」「这是一**（某）年**那个真题」（此处转写为「这是一年那个真体」）。
> 他给题目 **一律只报编号**：**例 3.4**（书上例题）、**30 题、第五题、第七题**（Cache 大题）、**第七题**（快表大题）、**15 题 / 20 题**，以及「书上的图 3.27 是一道真题的图」。
> 按 BRIEF 的要求「真题可标年份，但必须说明认定依据」——**本讲既没有老师给出的年份，本资料也没有可独立核实的题面来源**（他念的是口述版题干，不是原卷），因此**全部标为「年份不详」**，只保留他用的题号。若后续要标年份，必须回到原卷核对题干，不能靠「看着像」来认定。

| 题目 | 老师本讲的原话/编号 | 年份认定 |
| --- | --- | --- |
| 例 3.4（三问） | 「这个例题 3.4 可以做一下」「这个例题还出的比较好的」 | 年份不详 他称其为**书上例题**，未说年份 |
| 30 题 / 15 题 / 20 题（命中率、页表与快表选择题） | 「然后这个三十题」「做一下十五和二十」「快速的做下这两个选择（题）」 | 年份不详 只有题号，本资料不认定 |
| 第五题（数组地址 → 行号）<br>第七题（Cache 大题）<br>第七题（快表大题） | 「这个第五题这个大题……就不再展开了」「看一下这个第七题吧」「看一下第七题这个大题」 | 年份不详 只有题号；两处「第七题」编号重复（应属不同小节），本资料不据此换算年份 |
| 图 3.27 那道真题图<br>「快表的组相联」真题 | 「这是一道真题的图，里面几乎涉及到操作系统的东西，也涉及到（计算机）组成原理的东西」；「其实书上没有专门讲，但是出现在真题里面出现挺多的」 | 年份不详 他明确说是**真题**、且说**高频**，但**没说年份**；本资料不认定 |

## 6 易错点清单

| 容易搞错的地方 | 为什么错 | 正确做法 / 老师原话 |
| --- | --- | --- |
| 把「地址」和「地址里的数据」搞混 | 块里放的是**数据**，块号才是地址 | 「**很多人刚学的时候会把那个地址和数据搞混，这个一定要避免**」——10 号是地址，10 号里面的数据是 M |
| 以为 Cache 里的标记 = 主存块号 | 存的是**商**，不是块号 | 「标记的内容，其实他说相当于主存中的块的编号，**但是这里其实说的不太准**」；存商能反推块号且更省位 |
| 把「地址结构的标记」与「Cache 里的标记」当成一个 | 位数一样、**含义不同** | 「注意标记是**两个标记**……虽然位数什么都一样，但是含义是不一样的」；命中 = 两者相等 |
| 只看标记相等就判命中 | 还要看**有效位** | 「**标记相等并且有效位为一才是命中的**」；写操作只改主存没改 Cache 时，有效位要置 0 |

| 容易搞错的地方 | 为什么错 | 正确做法 / 老师原话 |
| --- | --- | --- |
| 直接映射也套替换算法 / 替换算法位一律写 $\log_2$ 行数 | 直接映射只有一个位置可选；位数要**看映射方式** | 「他的特点就**无需使用替换算法**」「就不存在多个被选项」；直接映射 **0 位**、全相联 **$\log_2$ 行数**、组相联 **$\log_2$ 路数** |
| 全写 / 直写也写一位脏位 | 直写没有脏位 | 「**直写意思就是说没有脏位**」「只有回写才有修改位」 |
| 算总容量时漏掉标记 / 有效位 | 「考试是不会给」这几位 | 每行 = 数据 + 标记 + 有效位 + 脏位 + 替换算法位；**数据给的是字节、标记是比特，要统一单位** |
| 忘记「读一次 + 写一次 = 2 次访问」 | 缺失率的分母变了 | 「**一个元素是访问两次的，就读一次写一次**」；`A[i]=A[i]+32` → 1/8，只读或只写 → 1/4 |
| 不管数据占不占整数个块就用「局部代表整体」 | 前提不成立 | 「**它是刚好放了整数个 Cache 块——250 个块**」；否则只能「总缺失次数 ÷ 总访问次数」 |
| 按字编址时仍按字节数位宽 | 块内地址位数由**地址个数**决定 | 「按字编址，一个字 32 位（4 字节）→ 64B 的块只占 16 个地址 → 块内地址 4 位」；「**按字编址一定要稍微细心一点**」 |
| 「说明访问过程」漏掉最后一步取数 | 命中 ≠ 任务完成 | 「**CPU 的目的只是为了取出其中一个块的某一个数据，不是为了整个块**」 |
| 题干说「初始 Cache 为空」还去写比较标记的过程 | 多答步骤（不扣分但浪费） | 「**初始时 Cache 空就意味着一定不会命中**」「没必要再这么回答」 |
| 把快表当内存 / 以为快表只有全相联 | 快表属于缓存；快表也有三种映射 | 「**快表注意，它不属于内存**，因为页表都是在内存放的」；「**无论是快表用的什么映射方式，前面通通都可以当成页号**」 |
| 块号 / 页框号 / 框号混用 | 「框号」在页表里指**页框号** | 「这个框号就是指的页框号，**不要和 Cache 那个块搞混了**」 |

## 7 考场速查卡

【一】必背的地址结构（高位 → 低位）
直接映射：标记 | 行号 | 块内地址　　行号 = 块号 mod 行数 ，标记 = 块号 ÷ 行数
全相联　：标记 |　　块内地址　　标记 = 整个块号
组相联　：标记 | 组号 | 块内地址　　组号 = 块号 mod 组数 ，标记 = 块号 ÷ 组数
快表同构：把「块号」换成「虚页号」，行/组数换成快表的行数 / 组数（完全通用）

【二】拆地址两步（十进制）
① 地址 ÷ 块大小（页大小） → 商 = 块号（虚页号），余数 = 块内地址（页内地址）
② 块号 ÷ 行数（直接）或 ÷ 组数（组相联） → 商 = 标记，余数 = 行号 / 组号
③ 二进制法：从低位往高位切 —— 块内地址 / 行号或组号 / 标记
⚠ 按字编址：块内地址位数 = $\log_2$(块大小 ÷ 字长)，不是 $\log_2$(块字节数)

【三】总容量
Cache 总容量 = 行数 × ( 数据块(字节×8) + 标记 + 有效位 + 脏位 + 替换算法位 ) 单位：比特
行数 = 数据区容量 ÷ 块大小 组数 = 行数 ÷ 路数
有效位：1 位（必有） 脏位：回写 1 / 全写 0 替换算法位：直接 0 / 全相联 $\log_2$行数 / 组相联 $\log_2$路数
考试不给这几位的含义与位数，必须自己记

【四】命中率 / 缺失率
命中 ⟺ 标记相等 且 有效位 = 1 ；命中后还要按块内地址取数
局部代表整体（占满整数个块）：缺失率 = 每块缺失次数 ÷ 每块访问次数
硬算（不占整数个块） ：缺失率 = 总缺失次数 ÷ 总访问次数
⚠️「读一次 + 写一次」= 2 次访问 ；块的首次访问必缺失

【五】访问过程答题（命中 / 缺失）
命中：定位行(组) → 比标记 → 标记等且有效位 1 → 按块内地址取数
缺失：整个块调入 → 写入标记 → 有效位置 1 → 按块内地址取数
题干说「初始 Cache 为空」→ 直接写缺失四步，不必写比较

【六】比较器与快表
比较器个数：直接 1 / 全相联 = 行数 / 组相联 = 路数 ；位数恒等于标记位数
Cache 对【物理地址】、存【数据】；快表对【逻辑地址】、存【页框号】；两者都是缓存、都不在内存
页表在内存 → 查表慢；快表在缓存 → 查表快；两者最终都是为了拿到页框号再与页内地址拼接

**如果只记六件事**

① **映射 = 把块号拆开**：直接拆成「标记 + 行号」，全相联整个是标记，组相联拆成「标记 + 组号」；**快表照搬**；
② **两个标记 + 一个有效位**：地址结构里的标记与 Cache 里的标记相等、且有效位为 1，才算命中；
③ **每行 = 数据 + 标记 + 有效位 + 脏位 + 替换算法位**，单位统一成比特；这几位考试不给；
④ **替换算法位**：直接 0 / 全相联 $\log_2$行数 / 组相联 $\log_2$路数；**写策略**：全写（直写）无脏位、回写有脏位；
⑤ **缺失率**先看占不占整数个块；`A[i]=A[i]+32` 是**读一次写一次**（1/8），只读或只写是 1/4；
⑥ **答题最后一步永远是按块内地址取数**；题干说「初始 Cache 为空」就省掉比较过程。

## 附：本资料的来源与边界

| 来源 | 内容 | 可靠性与边界 |
| --- | --- | --- |
| B 站《408 各科大题知识点强化课》课程第 6 讲「计算机组成原理 · Cache」，分 P8，音频 **3 小时 25 分 11 秒** | §1 – §7 的全部 | 本地 Whisper 转写，讲稿 `.md` 2646 行 / 约 13.0 万字节，覆盖 `[00:00:05]` – `[03:24:37]`，即**全程**；其中 `[01:44:26]` – `[01:53:55]` 是课间休息（约 9.5 分钟无内容），`[03:22:49]` 之后是课尾小结与下次课安排（已择要收录）。**本讲全程无板书**，地址结构、除法拆位、组号推演都是口述的，整理时按他念出的运算顺序还原，并把每一处算式独立重算过一遍（见下）。 |

| 来源 | 内容 | 可靠性与边界 |
| --- | --- | --- |
| 教材 / 通用常识（本资料的补充） | 「时间局部性 / 空间局部性」这两个名称（§2.2 的警告框里只作提示，未展开）；「FIFO / 随机」等其余替换算法的名称（仅作为「他没讲」的说明）；「写分配 / 非写分配」（仅作为「他没讲」的说明） | **教材补充，非老师口径**。这些术语在全讲中转写稿里出现 **0 次**，本资料**没有替他把内容补出来**，只在对应位置标注「本讲未讲」。 |
| 本资料的算式补算 | 例 3.4 的每行总容量 532 位 / 总容量 4256 位、3200 号地址的行号 2 与标记 6、Cache 大题缺失次数 64、§4.1 表里 5 个地址的取数位置 | **本资料补算，非老师原话**（他给的是方法与中间量，没有念出这些结果）。每一处都在正文里带了 ⚠ 标注，便于区分。 |

⚠ **转写含糊、按上下文推断或存疑的地方（逐条列出）**

① `[00:28:57]`–`[00:29:02]`「它对应的**框号 12**」「对应的框内地址也是**12**」→ 按同段算式（10 ÷ 4 = 商 2 余 2），应为**块号 2、块内地址 2**；「12」疑为转写重复（§4.1）。

② `[00:22:01]` 起他念的地址串与数据串（「89411」「XY、MN」「HIGK」）是现场画图时念的，本资料只采用**能与他自己的算式对上**的部分（10→M、18→C、17→B、3→K、1→I），其余不采用（§4.1）。

③ `[00:31:00]`「零内存的零号除以 4，它的商是 1，余数是 0」→ 按上下文应为**商 0**（0 ÷ 4 = 商 0 余 0）；同段 4 号块的「商是 1、余数也是 0」是对的。本资料按**商 0**取（§2.4）。

④ `[01:06:47]`「还有标记，**标记有 4 个**」与 `[01:26:14]`「这个标记（位）一共有四个」→ 指**每行一个、这一例共 4 行故 4 个**；例 3.4 换成 8 行时他不再提「几个」，本资料按「每行一个标记」理解（§2.9）。

⑤ `[02:18:50]`「总共 32 位扣掉 **12**（还是 **25**）是标记」→ 后半句无法辨读；按 32 − 6 − 6 = 20 与他紧接着说的「所以标记这 20 位」一致，本资料按 **20 位**取（§5.1）。

⑥ `[03:04:29]`「总共虚拟地址 30 位，扣掉了 15 还剩 **17** 位是标记」→ 30 − 15 = **15**，他随后自己改口为 15 位（§5.2 警告框）。

⑦ `[03:04:42]`「如果单纯问虚页号，那就高 **20** 位」→ 30 − 12 = **18**，他随后自己纠正为 18 位（§5.2 警告框）。

⑧ `[03:09:01]` 他念的余数串「余 2、余 4、余 0、余 7、余 2、余 4、余 0」→ 前 6 个（10、12、16、7、26、4 对 8 取余）完全正确；第 7 个按他后面逐项分析应为 **余 4**（12 mod 8 = 4），转写成了余 0。本资料按**前 6 个原样、第 7 个取 4**（§5.2）。

⑨ `[03:08:03]` 那串「十十二十六，然后七二六四，然后十二二十」转写极度混乱；按他后文逐项说的「第一个放到二组、12 放到 4 组、16 放到 0 组、7 放到 7 组、26 放到 2 组、4 放到 4 组、又访问 12、最后一个是 20」反推，序列应为 **10、12、16、7、26、4、12、20**（转写碎片里「十二」「二十」各出现一次）；其中**重复访问的「12」是按「接下来又访问十二」还原的**（§5.2）。

⑩ Cache 大题第二问里数组的首地址：他念作「给了一个（首）地址」「你可以把这个 **C0** 展开」「最后的 C0 展开你会发现他的块内地址是 0」→「C0」是十六进制末字节（C0 = 1100 0000，低 6 位全 0）。**完整地址他没念出来**，本资料只写「末字节 C0、低 6 位为 0」这一条（§5.1）。

⑪ [02:28:37]「所以总共就会缺**十二的六十八次**」→ 转写崩坏；按上下文应为「**2 的 6 次方次 = 64 次**」，且与方法一（2048 × 1/32 = 64）互验一致（§5.1）。
⑫ 20 题里他读出的标记值转写为「**03FFF**」→ 数值不可靠，本资料只写「整个高位都是标记」这一定性结论，不写具体值（§4.6）。
⑬ 15 题的题面数字（虚拟地址、页表项值）他念得很碎，只有「页面 4KB」「0x82 = 130」「存在位为 1」「最后选 C」是明确的，本资料只保留这四条（§4.6）。
⑭ 第五题他只说了「第一问让你算总容量」「第二问让你算这个行号」和两个数组地址公式，**没念 M、N、元素大小，也没念结果**，本资料不补（§4.5）。
⑮ 「第五题」与「第七题」的编号在本讲中出现过重复（Cache 大题与快表大题都被叫过「第七题」）→ 应属**不同小节各自编号**，本资料按「Cache 大题 / 快表大题」区分，不替他对号（§5.1、§5.2）。
⑯ 例 3.4 第三问他念的地址「**0123456**」进制不明 → 十进制 123456 与十六进制 0x123456 两种读法的行号都是 **001**，与他的说法一致；本资料只用行号 001（§4.2）。
⑰ 「4 号块数据 = ABCD」而访问 17（块内地址 1）取出的却是 **B**、访问 18（块内地址 2）取出 **C** —— 说明块内地址是从 0 开始编号的（0=A、1=B、2=C、3=D）。这一点他没有明说，是**本资料从三个取数结果反推出来的**（§4.1）。
⑱ 课尾一句「下次大概在这个桌上晚上到处」完全无法辨读，本资料未收录。

**老师没有讲 / 本讲没有覆盖的内容（对照你给的大纲）**

**① Cache 性能计算里的「平均访问时间」「加速比」「平均访存时间」「命中时间」「缺失代价」——本讲未讲（全讲 0 次）。**他只讲了命中率 / 缺失率（§2.10、§3.3），没有讲过「平均访问时间 = 命中时间 + 缺失率 × 缺失代价」这类公式，也没有出现过「加速比」。
**② 多级 Cache——本讲未讲（全讲 0 次）。**他没有提 L1 / L2 / L3，也没有提两级 Cache 的命中率与局部缺失率。
**③ 替换算法只讲了 LRU——FIFO、随机（RAND）本讲未讲（0 次）。**他只在「Cache 真题里用得最多的是 LRU」这一句上展开，并给了替换算法位的位数规则。
**④ 写策略只讲了全写（直写）/ 回写与脏位——「写分配 / 非写分配」本讲未讲（0 次）。**写缓冲、写直达对总线的影响也没讲。
**⑤ 「时间局部性 / 空间局部性」这两个名称本讲未讲（0 次）。**他只用了「局部性原理」四个字，并且只用一句话说明它的意思。
**⑥ 其他常规内容本讲未讲：**Cache 容量与块长如何权衡、多体交叉存储、Cache 一致性（MESI 之类）、指令 / 数据 Cache 分离的**原因**（他说「等到后面讲到流水线再展开」）、写缓冲；以及 TLB 的替换算法细节、TLB 与 Cache 同时缺失时的处理顺序、页表项的具体字段。
**⑦ 操作系统侧只讲到「会用」为止：**二级页表的完整查表流程、页框号与页内地址为什么拼接、缺页中断的处理，他都说「等到操作系统（课）到时候再展开来讲」「现在的话就会用就行了」。
**⑧ 无授课内容的时段：**课间休息 [01:44:26]–[01:53:55]；读题 [02:07:53]–[02:12:45]（约 5 分钟）。
**⑨ 本讲没有再讲一遍计组的分值分布：**关于「考什么」的判断只有 §1 引的那几句。

---

整合自 3 小时 25 分 11 秒网课讲稿（本地 Whisper 转写，无板书；地址结构、除法拆位与组号推演均按口述还原）· 未上传任何材料到第三方服务 · 全部例题、数值与算式均回对过讲稿原文并独立重算